Banking RAG Customer Support System

Module Design · Data Structures · Workflow Logic · API Contracts · Error Handling

Document Type	Low-Level Design (LLD)	**Codebase**	app.py + main.py
Primary Language	Python 3.10+	**Framework**	LangChain + Streamlit
Vector DB	ChromaDB (local)	**Primary LLM**	Groq llama-3.1-8b-instant
Embedding	all-MiniLM-L6-v2 (384-dim)	**Fallback LLM**	Falcon-RW-1B (HuggingFace)

Table of Contents

1. Module-Level Design Overview 3
2. Data Structures 4
3. Detailed Module Specifications 5
4. Workflow Design — LangGraph Node Model 8
5. Conditional Routing Logic 10
6. API / Interface Contracts 12
7. Error Handling Matrix 14
8. Configuration Reference 15

1. Module-Level Design Overview

The codebase is split into two Python files with clear separation of concerns: **app.py** owns all UI/session logic, and **main.py** owns all AI/data logic. This boundary ensures the AI pipeline can be tested independently of the Streamlit runtime.

1.1 app.py — Frontend & Session Layer

Function/Block	Responsibility
Import Resolution	sys.path.append(current_dir) ensures main.py is importable regardless of working directory.
Page Config	st.set_page_config sets title and centered layout once per session.
Sidebar	Renders 'Clear Chat' button; resets st.session_state.messages = [].
Session State	messages list persists across Streamlit reruns within the same browser tab.
Chat History Render	Iterates st.session_state.messages; uses role-specific avatars from Flaticon CDN.
Input Handler	st.chat_input() captures query; triggers ask_question(user_input) from main.py.
Response Renderer	Checks for ■ sentinel: st.error() for escalations, st.success() for AI answers.
State Update	Both user and assistant messages appended to session state after each exchange.

1.2 main.py — Backend RAG Engine

main.py executes in two phases: **Initialisation** (runs once at import time) and **Inference** (runs per query call).

- load_dotenv() → reads GROQ_API_KEY from .env
- Groq(api_key=...) → initialises Groq client
- pipeline('text-generation', model='tiiuae/falcon-rw-1b', ...) → loads Falcon weights into memory
- PyPDFLoader('banking_customer_support_qa.pdf').load() → Document list
- RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50).split_documents(documents) → chunks
- HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2') → embedding model
- Chroma.from_documents(chunks, embeddings, persist_directory='./db') → vectorstore
- vectorstore.as_retriever(search_kwargs={'k': 3}) → retriever

2. Data Structures

2.1 Document Object (LangChain BaseDocument)

```
# Produced by PyPDFLoader
Document(
    page_content: str,   # Raw extracted text from one PDF page
    metadata: dict       # {"source": "path/to/pdf", "page": int}
)
```

2.2 Chunk Object (post-split Document)

```
# Produced by RecursiveCharacterTextSplitter
Document(
    page_content: str,   # 500-char text window
    metadata: dict       # inherited from parent Document
)
```

2.3 Graph State (conceptual LangGraph StateDict)

```
# Represents the mutable state passed between nodes
GraphState = {
    "query":    str,            # Original user input
    "context":  str,            # Concatenated chunk text
    "response": str,            # LLM-generated answer
    "status":   "AI" | "HITL" # Routing outcome
}
```

2.4 Query-Response Contract (API boundary)

```
# Input to ask_question()
Input:  query: str
# Output from ask_question()
Output: response: str   # Either AI answer or escalation message
        # Escalation identified by '■' sentinel prefix
```

2.5 Groq API Payload

```
# Sent to Groq REST endpoint
{
    "model": "llama-3.1-8b-instant",
    "messages": [{"role": "user", "content": prompt}]
}
# Response path: data.choices[0].message.content.strip()
```

2.6 HITL Log Entry (human_requests.txt)

```
# Plain text, one query per line (append mode)
Format: <raw_query_string>\n
Example: 'My card was stolen, please block it immediately'
```

3. Detailed Module Specifications

3.1 Document Loader Module

Technology: PyPDFLoader

Input	File path: banking_customer_support_qa.pdf
Output	List[Document] — one Document per page
Method	PyMuPDF-based text extraction via LangChain community loader
Error Case	FileNotFoundError if PDF missing → application crashes at startup (fail-fast design)
Side Effect	None — read-only file operation

3.2 Chunking Module

Technology: RecursiveCharacterTextSplitter

Input	List[Document] from loader
Output	List[Document] — chunked, typically 5–20× more documents than input
chunk_size	500 characters — chosen to fit within MiniLM-L6-v2's 256-token window
chunk_overlap	50 characters — preserves sentence continuity across boundaries
Splitters Used	Tries in order: \n\n → \n → ' ' → '' (recursive fallback)
Metadata	Inherited from parent Document unchanged

3.3 Embedding Module

Technology: HuggingFaceEmbeddings

Model	sentence-transformers/all-MiniLM-L6-v2
Vector Dim	384 float32 values per chunk
Device	CPU (no GPU required)
Inference	Batch encode at startup; single encode per query at retrieval time
Normalisation	L2-normalised (cosine similarity = dot product)

3.4 Vector Store Module

Technology: ChromaDB (via LangChain)

Backend	SQLite + HNSWLIB (Chroma default)
Persistence	./db directory written on Chroma.from_documents()

Collection	Single default collection; metadata filtered by source/page
Index Type	Approximate Nearest Neighbour (HNSW) — M=16, ef_construction=100
Query Time	O(log n) with HNSW; < 10 ms for < 50k vectors

3.5 Retrieval Module

Technology: Chroma.as_retriever()

Strategy	Similarity search (cosine distance)
k	3 — returns top-3 most relevant chunks
Score Threshold	None set (returns k regardless of score)
Context Build	'\n\n'.join([doc.page_content for doc in docs])
Empty Guard	if not docs: return 'I don\'t know.' — prevents empty-context LLM calls

3.6 Prompt Construction Module

Technology: ask_question() inline

Template	Role instruction + Context block + Question + 'Answer:' suffix
Role Instruction	'You are a banking customer support assistant.'
Constraints	Answer in 2-3 lines; do not generate questions; do not repeat context
Injection Risk	Context is directly interpolated — no sanitisation (future: escape special tokens)

3.7 LLM Inference Module

Technology: Groq + Falcon fallback

Primary Call	groq_client.chat.completions.create(model, messages)
Response Parse	response.choices[0].message.content.strip()
Fallback Trigger	Any Exception from Groq call (network, quota, auth)
Fallback Call	fallback_llm(prompt)[0]['generated_text']
Fallback Cleanup	result.replace(prompt, '').strip() — removes echoed prompt
Determinism	do_sample=False ensures greedy decoding on fallback

3.8 HITL Detection Module

Technology: needs_human() + log_human_request()

Tier 1 — Exact	fraud | unauthorized | stolen | hack → True immediately
Tier 2 — AND	('card' in query) AND ('lost' OR 'block' in query) → True

Case Handling	query.lower() applied before matching
Log Action	open('human_requests.txt', 'a').write(query + '\n')
Return Value	'■ This query has been escalated...' (fixed string)
False Positive	e.g. 'I lost my account number' — 'lost' without 'card' → NOT escalated

4. Workflow Design — LangGraph Node Model

The system implements a conceptual three-node LangGraph pipeline. While the current codebase uses LangChain LCEL and direct function calls, the architecture maps cleanly onto LangGraph's StateGraph construct for future migration.

INPUT NODE	• Receives raw query string from Streamlit. • Initialises GraphState: {query: q, context: '', response: '', status: 'PENDING'}. • Passes state to ROUTING EDGE.
ROUTING EDGE (conditional)	• Evaluates needs_human(state['query']). • If True → transitions to HITL NODE. • If False → transitions to PROCESSING NODE.
HITL NODE	• Logs query to human_requests.txt. • Sets state: {response: escalation_msg, status: 'HITL'}. • Transitions to OUTPUT NODE immediately (bypasses LLM).
PROCESSING NODE	• Invokes retriever.invoke(state['query']) → docs. • Builds context string from doc.page_content. • Assembles prompt template with context + query. • Calls Groq API (or fallback) → answer string. • Sets state: {context: ctx, response: answer, status: 'AI'}.
OUTPUT NODE	• Reads state['response'] and state['status']. • Returns response string to Streamlit app.py. • Streamlit renders based on ■ sentinel in response.

4.1 State Transition Table

Stage	Status Field	State Description
Initial	PENDING	query set, others empty
Post-HITL Check (True)	HITL	response = escalation_msg
Post-HITL Check (False)	AI (in-progress)	context being built
Post-Retrieval	AI (in-progress)	context populated
Post-LLM (success)	AI	response = Groq answer

Post-LLM (fallback)	AI	response = Falcon answer (stripped)
Output	FINAL	response returned to caller

5. Conditional Routing Logic

The routing logic in needs_human() implements a two-tier keyword matching strategy. The decision tree below documents every branch with its corresponding code path.

```
def needs_human(query: str) -> bool:
    query = query.lower()            # Case-normalise
    # TIER 1 — HIGH-SEVERITY SIGNALS (single keyword sufficient)
    if any(word in query for word in
           ["fraud", "unauthorized", "stolen", "hack"]):
        return True                  # Immediate escalation
    # TIER 2 — COMBINATION SIGNALS (context-dependent)
    if "card" in query and any(word in query
            for word in ["lost", "block"]):
        return True                  # Card + loss/block → escalate
    return False                     # Safe for AI processing
```

5.1 Routing Decision Matrix

Sample Query	Match Path	Routing Decision
'my account was hacked'	Tier 1: 'hack'	HITL ✓
'report unauthorized transaction'	Tier 1: 'unauthorized'	HITL ✓
'I think there is fraud on my acc'	Tier 1: 'fraud'	HITL ✓
'my card was stolen'	Tier 1: 'stolen'	HITL ✓
'please block my card'	Tier 2: card+block	HITL ✓
'I lost my card'	Tier 2: card+lost	HITL ✓
'I lost my account password'	No card → Tier 2 miss	AI ✓
'how do I check my balance?'	No keywords match	AI ✓
'what are your loan rates?'	No keywords match	AI ✓
'block my internet access'	card not present	AI ✓

5.2 Known Limitations & Mitigations

Issue	Detail & Mitigation
False Negative Risk	Paraphrased fraud queries ('someone is using my account without permission') bypass Tier 1. Mitigation: NLU-based intent classifier in v2.
False Positive Risk	Low — Tier 2 requires AND logic. Tier 1 keywords are highly domain-specific.

Multilingual Blind Spot	Non-English queries bypass keyword detection entirely. Mitigation: translate-before-match in v2.
Case Sensitivity	Handled — query.lower() applied before all matching.

6. API / Interface Contracts

6.1 ask_question() — Primary Interface

```
def ask_question(query: str) -> str:
    '''
    Parameters
    --------------
    query : str
        Raw natural-language question from the user.
    Returns
    ----------
    str
        AI-generated answer (2-3 lines) OR
        '■ This query has been escalated...' for HITL cases.
        '"I don\'t know."' if retriever returns empty results.
    '''
```

6.2 Groq API Contract

```
# Request
POST https://api.groq.com/openai/v1/chat/completions
Headers: { Authorization: Bearer <GROQ_API_KEY> }
Body: {
    "model": "llama-3.1-8b-instant",
    "messages": [{"role": "user", "content": <prompt>}]
}
# Response (success path)
response.choices[0].message.content  ->  str (answer text)
# Error path
Any exception  ->  fallback to Falcon-RW-1B pipeline
```

6.3 Streamlit ↔ main.py Contract

app.py imports	from main import ask_question
Call signature	response: str = ask_question(user_input: str)
Escalation signal	'■' in response → st.error(); else → st.success()
Session persistence	st.session_state.messages: List[Dict[str, str]]
Clear chat	st.session_state.messages = [] on button click

6.4 HITL Log File Contract

```
File:     human_requests.txt
Mode:     append ('a') — never overwrites
Encoding: system default (UTF-8 recommended)
```

```
Format:   one raw query string per line
Consumer: human agent reads file manually (v1)
          future: dashboard polls file via inotify/webhook
```

7. Error Handling Matrix

The system implements a three-tier resilience strategy: fail-fast at startup, graceful degradation at LLM inference, and informative fallback for retrieval misses.

Error Scenario	Phase	Exception Type	Current Behaviour	Resolution/Note
PDF File Missing	Startup	FileNotFoundError	Application crash with stack trace	Fail-fast — config error must be fixed before serving
Embedding Model Load Fail	Startup	OSError / HTTPError	Application crash	Model cached after first download; check HF connectivity
ChromaDB Corrupt	Startup	Exception	Application crash	Delete ./db and restart to re-ingest
Groq API — Auth Error	Inference	AuthenticationError	Caught → Falcon-RW-1B activated	Verify GROQ_API_KEY in .env
Groq API — Rate Limit	Inference	RateLimitError	Caught → Falcon-RW-1B activated	Implement exponential backoff in v2
Groq API — Network Timeout	Inference	requests.Timeout	Caught → Falcon-RW-1B activated	Circuit breaker pattern recommended
Empty Retrieval (k=0)	Inference	docs == []	Return 'I don\'t know.' immediately	Tunable: lower similarity threshold or increase k
Streamlit Runtime Error	UI	Exception (any)	st.error(f'Error: {str(e)}') displayed	Error message appended to session state
HITL Log Write Fail	Routing	IOError	Unhandled in v1 — silent failure	Wrap log_human_request() in try/except in v2

8. Configuration Reference

8.1 Environment Variables (.env)

Variable	Type	Status	Description
GROQ_API_KEY	str	Required	Groq cloud API key for llama-3.1-8b-instant

8.2 Tunable Parameters in main.py

Parameter	Default Value	Location	Tuning Note
chunk_size	500	RecursiveCharacterTextSplitter	Increase for longer context windows
chunk_overlap	50	RecursiveCharacterTextSplitter	Increase to reduce boundary loss
k (retrieval)	3	as_retriever(search_kwargs)	Increase for broader context, decrease for precision
max_new_tokens	150	Falcon fallback pipeline	Increase for longer fallback answers
do_sample	False	Falcon fallback pipeline	Set True for creative/diverse answers
model (Groq)	llama-3.1-8b-instant	groq_client.chat.completions	Swap for llama-3.1-70b for higher quality
persist_directory	./db	Chroma.from_documents()	Change path to mount external volume

8.3 HITL Keyword Configuration

Keywords are hardcoded in needs_human(). To add/remove triggers without code changes, extract to a YAML config file in v2:

```yaml
# Proposed hitl_config.yaml (v2)
tier_1_keywords:
  - fraud
  - unauthorized
  - stolen
  - hack
tier_2_anchor: card
tier_2_modifiers:
  - lost
  - block
  - stolen   # overlaps with tier_1 for belt-and-suspenders
```